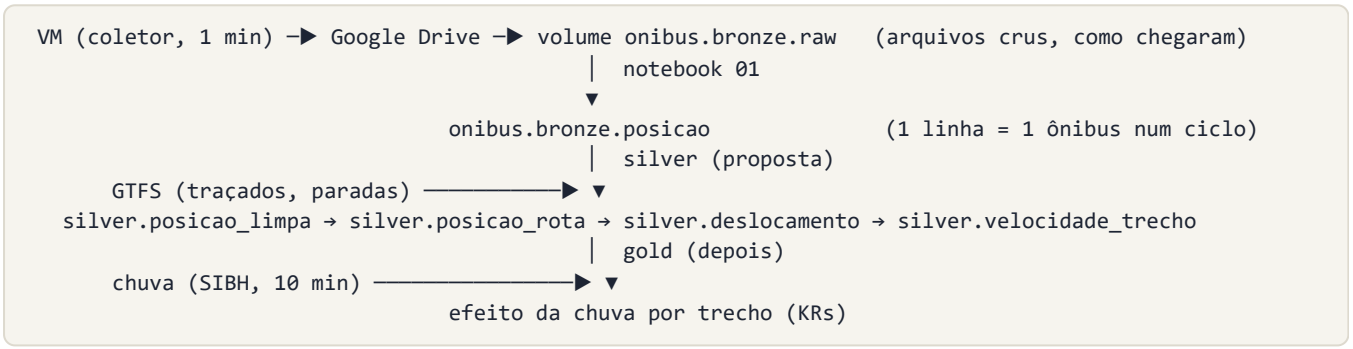
Bronze e silver

Como a bronze está estruturada e como proponho a silver

Projeto Ônibus SP × Chuva · Gabriel Graciano Dias
05/10/2026

> 05/10/2026. Exemplos tirados da tabela real `onibus.bronze.posicao`. Horários das tabelas em **UTC** (como estão guardados); entre parênteses, o horário de SP (UTC−3).

Resumo em uma figura



A regra de cada camada:

Camada	Pergunta que responde	O que não faz
Bronze	"O que a API disse?" — fiel à fonte	não limpa, não corrige, não junta com nada
Silver	"O que aconteceu de fato?" — limpo e enriquecido	não responde à pergunta do projeto
Gold	"Qual o efeito da chuva?" — pronto para os KRs	—

Parte 1 — A bronze hoje

1.1 Duas peças: arquivos e tabela

Volume `onibus.bronze.raw` — os arquivos exatamente como o coletor gravou, organizados por dia de SP:

```
/Volumes/onibus/bronze/raw/olhovivo/
├── auditoria.csv                          ← 1 linha por ciclo do coletor (o "batimento cardíaco")
├── bronze_posicao/                        ← formato v1 (25/08 → 21/09)
│   ├── dt=2026-08-25/  131 arquivos
│   ├── dt=2026-09-10/  663 arquivos       ← ~1 a cada 2 min
│   └── ...
└── bronze_raw/                            ← formato v2 (21/09 → hoje)
    ├── dt=2026-09-29/  1.437 arquivos     ← ~1 por minuto
    │   ├── posicao_20260929T110101Z.json.gz
    │   └── ...
    └── ...
```

Tabela `onibus.bronze.posicao` — os mesmos dados, abertos numa tabela Delta, particionada por `dt`.

Período	25/08 → 03/10 (40 dias)
Linhas	**262,3 milhões** (v1: 122,5 M · v2: 139,8 M)
Checagem	linhas por dia = soma de `veiculos_gravados` da auditoria → **dif = 0 em todos os dias**
Atualização	job `pipeline-onibus`, todo dia às 10h (SP), para o dia anterior

1.2 Os dois formatos de arquivo

v1 (até 21/09) — o coletor já achatava: cada linha do arquivo é um ônibus.


```
{"ta": "2026-09-10T03:00:56Z", "p": "36694", "cl": 2144, "sl": 1, "c": "4051-10",
 "lt0": "CPTM GUAIANASES", "lt1": "JD. SÃO PAULO", "py": -23.563019, "px": -46.401088,
 "a": true, "hr": "00:01", "coletado_em": "2026-09-10T03:01:03+00:00"}
```


v2 (desde 21/09) — o coletor guarda a resposta inteira da API, sem mexer. Ela é aninhada: linhas de ônibus → veículos.


```
{"hr": "08:01",
 "l": [ {"c": "507T-10", "cl": 34976, "sl": 2, "lt0": "METRÔ CARRÃO", "lt1": "TERM. SAPOPEMBA", "qv": 6,
         "vs": [ {"p": 48874, "a": true, "ta": "2026-09-29T11:00:22Z", "py": -23.6060545, "px":
         -46.503138},
                 ... ] },
        ... ] }
```


O notebook 01 lê os dois, abre o v2 com dois `explode` (resposta → linhas → veículos), e junta tudo com `unionByName`.

1.3 O schema da tabela

Coluna	Tipo	O que é	Exemplo
`dt`	string	dia de **SP** do ciclo (partição)	`2026-09-29`
`coletado_em`	timestamp	quando **a VM** recebeu (UTC)	`2026-09-29 11:01:01`
`ta`	timestamp	quando **a SPTrans** registrou a posição (UTC)	`2026-09-29 11:00:22`
`p`	bigint	prefixo: **o ônibus físico**	`48874`
`c`	string	letreiro = `route_id` do GTFS	`507T-10`
`cl`	bigint	código interno linha+sentido	`34976`
`sl`	int	sentido (1 ou 2)	`2`
`py`, `px`	double	latitude, longitude	`-23.6060545`, `-46.503138`
`a`	boolean	acessível	`true`
`lt0`, `lt1`	string	os dois terminais da linha	`METRÔ CARRÃO`, `TERM. SAPOPEMBA`
`hr`	string	hora da resposta (SP)	`08:01`
`arquivo`	string	caminho do arquivo de origem	`.../posicao_20260929T110101Z.json.gz`
`formato`	string	`v1` ou `v2`	`v2`

Os três campos que sustentam o projeto: `p`, `ta` e `py`/`px`. O mesmo `p` ao longo de vários `ta` é uma trajetória. `c` e `sl` fazem a ponte com o GTFS.

1.4 Um exemplo real: o ônibus 48874 chegando ao terminal

Linha 507T (Metrô Carrão ↔ Terminal Sapopemba), 29/09, das 8h00 às 8h09 de SP:

[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
11:01:01	11:00:22 (08:00)	2	34976	−23.60605	−46.50314
11:02:02	11:01:52 (08:01)	2	34976	−23.60955	−46.50136
11:03:01	11:02:37 (08:02)	2	34976	−23.61200	−46.50138
11:04:02	11:03:58 (08:03)	2	34976	−23.61391	−46.50091
11:05:02	11:04:50 (08:04)	**1**	**2208**	−23.61412	−46.50123
11:06:01	11:05:35 (08:05)	1	2208	−23.61412	−46.50123
11:07:01	11:06:20 (08:06)	1	2208	−23.61412	−46.50123
11:08:02	11:07:22 (08:07)	1	2208	−23.61412	−46.50123
11:09:02	11:08:48 (08:08)	1	2208	−23.61412	−46.50123

O que dá para ler aqui — e cada item vira uma regra da silver:

1. **O ônibus anda para o sul** nas 4 primeiras linhas (latitude ficando mais negativa) — indo para o Terminal Sapopemba no sentido 2.
2. **Às 08h04 chega ao terminal e troca de sentido**: `sl` 2 → 1, `cl` 34976 → 2208. A partir daí ele está "na linha do sentido 1", mas **parado**.
3. **Parado no terminal, com `ta` mudando**: a posição é idêntica por 5 minutos, mas o `ta` avança. Não é duplicata — é um ônibus ocioso. Se entrar no cálculo, vira "velocidade 0 km/h" e puxa a média do trecho para baixo **sem ter nada a ver com chuva**.
4. **Dois relógios**: `ta` (fonte) vem 4 a 41 s antes de `coletado_em` (VM). Para velocidade, usamos sempre `ta`.
5. **O intervalo entre `ta`s não é fixo** (90 s, 45 s, 81 s, 52 s): a fonte atualiza a cada ~45 s e nós amostramos a cada 60 s.

Velocidades entre os pings (distância em linha reta ÷ tempo):

De → até (SP)	Distância	Tempo	Velocidade
08:00:22 → 08:01:52	429 m	90 s	17,2 km/h
08:01:52 → 08:02:37	272 m	45 s	21,8 km/h
08:02:37 → 08:03:58	218 m	81 s	9,7 km/h
08:03:58 → 08:04:50	41 m	52 s	2,8 km/h ← chegando no terminal

Note que **cada par anda 200–430 m**. Um trecho entre paradas tem ~300–400 m. Por isso, na silver, o par de pings — e não o ping — é a unidade de observação.

O mesmo ônibus no v1 (10/09, 2 em 2 min) — mesmo trajeto, metade da resolução:

[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
08:37:32	08:37:19	2	−23.60767	−46.50186
08:39:42	08:39:00	2	−23.61350	−46.50161
08:41:52	08:41:15	2	−23.61394	−46.50121
08:44:03	08:43:31	1	−23.61412	−46.50123

1.5 O que a bronze tem de “sujeira” (de propósito)

Fenômeno	Tamanho	Por que está lá
Duplicatas `(p, ta)`	**0,58%** no v2; ~0% no v1	O v2 não deduplica (o v1 deduplicava no coletor). A fonte às vezes não atualiza entre dois ciclos
Ônibus parado no terminal/garagem	? (medir)	É o que a API mostra
Troca de sentido no terminal	toda viagem	Idem
Ciclos perdidos	~1,3–1,6%	163 + 118 respostas `null` (descartadas), falhas de login
Linhas especiais (`PAEC-10`, `TREI-10`)	3 veículos	Treinamento/serviço especial
Diferença de resolução v1 (2 min) × v2 (1 min)	—	Mudança de cadência em 21/09

Nada disso é corrigido na bronze: se um filtro da silver estiver errado, dá para refazer a partir dela.

Parte 2 — A silver proposta

2.1 O objetivo

Transformar **"onde cada ônibus estava a cada minuto"** em **"a que velocidade os ônibus andaram em cada trecho entre paradas, a cada 10 minutos"** — o insumo dos KRs 1–4.

Quatro tabelas, em sequência. Cada uma tem **uma checagem de contagem** (entra = sai + descartado, com motivo), como a bronze.

2.2 Antes: o GTFS

O GTFS da SPTrans (baixar no site de desenvolvedores; subir para o volume) vira tabelas `silver.gtfs_*`. O que usamos dele:

Arquivo	Para quê	Exemplo
`routes.txt`	`route_id` = `c` da API (99,8% casam)	`507T-10`
`trips.txt`	linha + `direction_id` → `shape_id`	`507T-10`, 0 → `shape 12345`
`shapes.txt`	o traçado: pontos com **distância acumulada** (`shape_dist_traveled`)	ponto 87 está no metro 4.210
`stops.txt` + `stop_times.txt`	as paradas, em ordem, de cada viagem	parada 3 da 507T sentido 0
`frequencies.txt`	headway programado (medida secundária)	6 min no pico

Validações já feitas (agosto): `sl=1 → direction_id=0`, `sl=2 → direction_id=1`; posições já vêm encaixadas no traçado (erro lateral 0,00 m em 99,8%).

2.3 Etapa 1 — `silver.posicao_limpa`

Faz:

- Remove duplicatas `(p, ta)`.
- Remove impossíveis: fora da caixa de SP (lat −24,01 a −23,35; lon −46,83 a −46,36), `ta` depois de `coletado_em`, posição com mais de 5 min de idade.
- Remove `PAEC-10` e `TREI-10`.
- Converte `ta` para horário de SP (`ta_sp`) — dia útil × fim de semana × feriado (07/09!) dependem disso.
- Junta com o GTFS: `c` → `route_id`, `sl` → `direction_id`, → `shape_id`. Linhas sem shape vão para a quarentena com motivo.

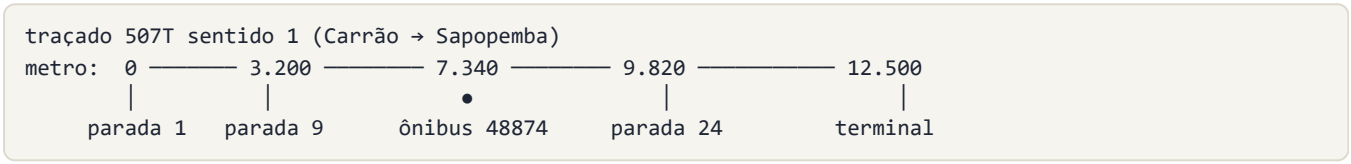
Exemplo (o 48874): as 9 linhas passam, ganhando colunas.

48874	08:00:22	507T-10	1	S_507T_1	−23.60605	−46.50314
…						
48874	08:04:50	507T-10	**0**	**S_507T_0**	−23.61412	−46.50123

Checagem: `bronze (do dia) = posicao_limpa + Σ descartes`, com uma tabela `silver.quarentena` contando por motivo.

2.4 Etapa 2 — `silver.posicao_rota` (a mais difícil)

A ideia: o mapa 2D vira uma régua 1D. Cada posição é projetada no traçado da sua linha e sentido, e vira "o ônibus está no metro *d* da rota". As paradas também são projetadas na mesma régua.



Exemplo:

			(metro da rota)
48874	08:00:22	S_507T_1	11.630
48874	08:01:52	S_507T_1	12.060
48874	08:02:37	S_507T_1	12.330
48874	08:03:58	S_507T_1	12.500 ← fim da rota
48874	08:04:50	S_507T_0	0 ← começo da volta

(valores de `d_m` ilustrativos — sairão da projeção real)

Como projetar — duas opções:

	(a) Python,	(b) SQL espacial do Databricks
Como	`LineString(traçado).project(Point(px, py))` dentro de `applyInPandas`, um grupo por (shape × dia)	picar o traçado em pedaços curtos com distância acumulada; para cada ping, o pedaço mais próximo (`st_dwithin` / `NEAREST BY`) + offset com `st_closestpoint`
Prós	a projeção é uma linha; fácil de explicar e testar	aprende o recurso espacial nativo (GA em jun/2026)
Contras	limite de 1 GB de memória por grupo no serverless	não existe função pronta de "metro da rota"; mais SQL

Recomendação: (a) primeiro. Projetar em metros exige converter lat/lon para um plano (a silver antiga usa uma equirretangular centrada em SP; erro < 0,1% em 50 km).

Cuidados:

- Traçado que passa duas vezes pelo mesmo lugar (circulares, ida e volta na mesma avenida): a projeção pode cair no ponto errado. Mitigação: usar o shape do **sentido certo** e conferir que `d_m` cresce ao longo da viagem.
- Linha + sentido com mais de um shape (variantes): o código já avisa quantos casos há; medir antes de decidir.

2.5 Etapa 3 — `silver.deslocamento`

Para cada ônibus, ordenar por `ta` e formar **pares de posições consecutivas** (`LAG` numa janela por `p`):

[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	Δd	Δt	v (km/h)	situação
48874	08:00:22	08:01:52	11.630	12.060	430	90	17,2	ok
48874	08:01:52	08:02:37	12.060	12.330	270	45	21,6	ok
48874	08:02:37	08:03:58	12.330	12.500	170	81	7,6	ok
48874	08:03:58	08:04:50	12.500	0	—	—	—	**troca de viagem** (cortado)
48874	08:04:50	08:05:35	0	0	0	45	0	**parado no terminal**
…								

Regras (cada uma com contagem de descartes):

Regra	Por quê
Δt > 5 min → descarta	buraco na coleta; interpolar seria inventar
mudou `shape_id` ou `direction_id` → corta	é uma viagem nova, não um deslocamento
Δd < 0 pequeno (~0,3%) → descarta	ruído/andou para trás
ônibus parado perto do início/fim da rota → descarta	ocioso no terminal (o exemplo acima)
v fora de 2–80 km/h → descarta	implausível (mesmos limites do artigo de Niterói)

Velocidade aqui é **comercial**: inclui paradas em ponto e semáforo — é o que o passageiro sente, e é o que comparamos com/sem chuva.

2.6 Etapa 4 — `silver.velocidade_trecho`

Trecho = de uma parada à seguinte, na régua da rota.

O problema: um par anda 200–430 m; um trecho tem ~300–400 m. Então a maioria dos trechos **não tem ping dentro** — não dá para medir "tempo dentro do trecho" diretamente.

A solução: rateio. Cada par distribui sua distância e seu tempo entre os trechos que atravessou, proporcionalmente (supondo velocidade constante dentro do par):

```
par: d 12.060 → 12.330 (270 m) em 45 s

         parada 26 (12.150)
               │
  12.060 ──────┼──────────────── 12.330
     90 m (15 s) │   180 m (30 s)
   trecho 25→26   trecho 26→27
```

Depois soma por trecho × janela de 10 min:

[illegible]	trecho	[illegible] → [illegible]	janela (SP)	Σ distância	Σ tempo	velocidade	nº de pares	nº de ônibus
S_507T_1	25→26	410012 → 410013	08:00–08:10	1.840 m	410 s	**16,2 km/h**	9	4
S_507T_1	26→27	410013 → 410014	08:00–08:10	2.150 m	395 s	**19,6 km/h**	10	4

(números ilustrativos)

Decisões embutidas:

- **Janela de 10 min** — casa com os pluviômetros do SIBH.
- **Velocidade = Σ distância ÷ Σ tempo** (média harmônica ponderada), não a média das velocidades — senão um par lento e curto pesa igual a um longo.
- **Trecho físico:** guardar `(parada_de, parada_ate)` além de `(shape_id, trecho)`. A mesma rua com 10 linhas vira **um trecho físico com 10× mais dados** — é o que dá estabilidade ao ranking do KR3.
- **Nº de pares e de ônibus** ficam na tabela: na gold, trechos com pouca observação são filtrados (o "[N] como regra" dos KRs).

2.7 Como vamos construir

1. **Baixar o GTFS** e subir para o volume; tabelas `silver.gtfs_*`.
2. **Uma linha e um dia** (ex.: 507T em 29/09) — as 4 etapas num notebook.
3. **Conferir visualmente:**
   - a trajetória de um ônibus no mapa;
   - o **diagrama de Marey** (metro da rota × hora, uma linha por ônibus) — ônibus parados aparecem como trechos horizontais, viagens como diagonais;
   - a velocidade por trecho ao longo do dia (deve cair no pico, 6–9h e 17–19h — também testa o fuso).
4. **Escalar** para todas as linhas de um dia, depois todos os dias — por partição `dt`, com `replaceWhere` (idempotente).
5. Entrar no job: `ingestao_drive` → `bronze_posicao` → `silver_*`.

2.8 Decisões em aberto

Decisão	Opções	Recomendação
Método de projeção	`shapely` / SQL espacial	`shapely` para começar
Janela temporal	5 / 10 / 15 min	10 min (pluviômetro)
Linha de teste	507T / 2290 / outra de corredor	uma de alta frequência num corredor
Resolução v1 × v2	usar tudo / subamostrar v2 para 2 min	análise principal com tudo; checar sensibilidade
Shapes múltiplos	o primeiro / tratar variantes	medir quantos casos antes

2.9 O que fica para a gold

- Juntar **chuva** ao trecho × janela (posto mais próximo ou interpolação; H3 ajuda).
- **Linha de base** do trecho × hora × tipo de dia, e o desvio em relação a ela (KR1).
- Eventos de chuva e **tempo de recuperação** (KR2).
- **Ranking** com bootstrap por dia (KR3); associação com tipo de via e alagamento (KR4).
- Headway como medida secundária.